Esercizio 2026/10/27

Angelo Petrignano, matricola 31288A

October 2026

1 Esercitazione

1.1 Spiegazione della soluzione

La soluzione da me individuata prima di tutto ordina l'array in modo decrescente, infatti dovendo fare dei confronti per trovare due numeri appartenenti all'array tali che sommati ne diano un altro dell'array, in questo modo si evita di considerare come papabili a e b dei numeri maggiori di c, che sappiamo già che se sommati con qualsiasi numero non negativo (per ipotesi) non potranno mai dare la nostra c. Successivamente si parte dal primo elemento, il quale diventa la c, il numero successivo diventa la a, in questo primo ciclo si scorrono gli elementi successivi alla a, i quali sono la b, se nel ciclo si ha che $a + b = c$ allora abbiamo trovato i due numeri, ed il l'intera funzione termina con esito positivo, se invece non si è trovata una b che rispetta tale condizione si "porta avanti" la a, cioè semplicemente si considera come a l'elemento dell'array successivo alla precedente. Se si è arrivati alla fine certamente non esistono a, b, c che rispettino la condizione della consegna, quindi la funzione ritorna al chiamante con il valore *false*.

1.2 pseudocodice

Di seguito è riportato lo pseudocodice di quanto spiegato in precedenza:

```
quick_sort(S);

flag = false;
for i = o to n-1
{
  c = S[i];
  j = i+1;
  a = S[j];
  for j + 1 to n
  {
    if a + S[j] == c then return true;
```

```
  }
}
return false;
```

1.3 Discussione sulla complessità

Per ogni assegnamento di b viene eseguito un numero di confronti con gli elementi successivi dell'array nell'ordine di n; tale numero di confronti viene reiterato per ogni assegnamento diverso di b, il quale viene eseguito a sua volta n volte per ogni assegnamento di a, questo ultimo assegnamento viene eseguito all'incirca n volte, pertanto la complessità finale risulta essere $O(n^3)$ La complessità di certo non è soddisfacente, l'unico vantaggio individuabile in tale esecuzione dell'algoritmo è che vengono evitati confronti "inutili", cioè con numeri che sappiamo che a priori che sommati non possono in nessun caso dare il valore c. Si nota che, data l'elevata complessità, applicare l'algoritmo di ordinamento non ha portato al minimo peggioramento della complessità dell'algoritmo (il *quick sort* ha infatti complessità $O(nlogn)$); non si riporta una distinzione di complessità tra caso peggiore, migliore e medio, perchè:

- Il caso migliore capita molto raramente (la probabilità è $\frac{1}{n}$);
- Il caso medio e peggiore differiscono solamente di valori costanti.

2 Problemi proposti a lezione

2.1 Problema 1

La soluzione di tale problema ha alla base l'utilizzo di una struttura dati di tipo pila, nello specifico:
Si crea una pila vuota, si scorre ogni carattere della stringa, quando si incontra una parentesi tonda aperta (il carattere "(") si aggiunge un elemento alla pila, quando si incontra una parentesi tonda chiusa (il carattere ")") si toglie un elemento dalla pila, se si cerca di togliere un elemento da una pila vuota significa che è presente una chiusura di parentesi prima che essa venga aperta, e può ritornare un messaggio di errore di questo tipo, se invece si arriva alla fine della stringa e la dimensione è strettamente maggiore di 0 allora qualche parentesi non è stata chiusa, e si riporta un messaggio di errore adeguato; l'ultimo caso che rimane è che la dimensione della pila sia 0, in quel caso le parentesi sono bilanciate e ben annidate, perciò la funzione ritorna con esito positivo.